
Table of Contents

.....	1
1. Construction du jeu de données	1
2. Simulation du filtre gaussien	2
5. Simulation du problème direct	3
6. Simulation du bruit de quantification	4
7.Calcul du bruit de quantification	5
8.Longueurs des signaux d'entrée et de la réponse impulsionnelle	6
9.Dimensions	6
10.Calcul de la sortie non bruitée par opération matricielle	7
11. Déconvolution naïve et constat d'instabilité	8
2. Déconvolution	9
12. Base de temps pour le signal reconstruit	9
13.Initialisation des paramètres	10
14.Construction de la matrice de convolution H	12
15. Tracé du signal original et reconstruit	15
16.Calcul des valeurs singulières de la matrice de convolution H	17
17.Initialisation des paramètres	17
Vérification que x est un vecteur colonne	19
20.éconvolution par régularisation quadratique	20

```
%%-----  
%%--- Fichier : ExamTP.m  
%%--- Objet : Devoir TP  
%%--- Auteur :Salma EL ARJA  
%%--- Date : 04/02/2025  
%%-----
```

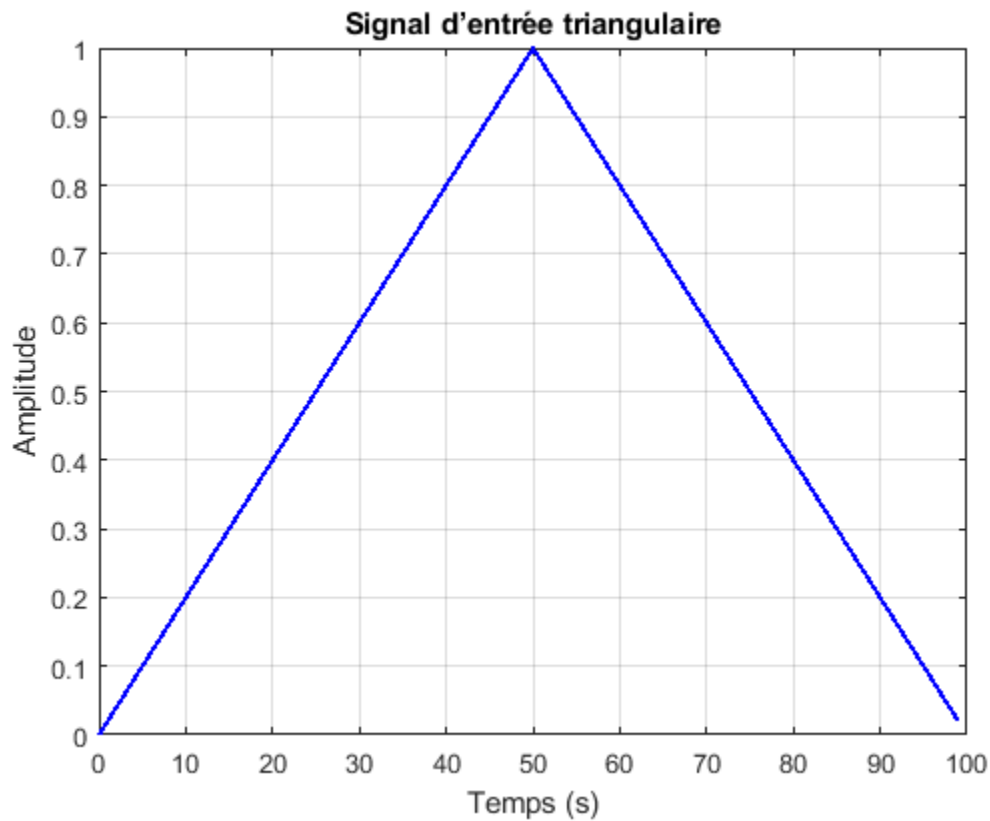
```
clear all;  
close all;  
clc;
```

1. Construction du jeu de données

Paramètres de base

```
L = 100; % Durée d'observation (secondes)  
fe = 1; % Fréquence d'échantillonnage (Hz)  
Te = 1/fe; % Période d'échantillonnage (secondes)  
Nx = L/Te; % Nombre d'échantillons du signal x  
nx = 0:Nx-1; % Indices temporels  
  
tx = nx * Te; % Base de temps (secondes)  
  
% Simulation du signal d'entrée triangulaire  
x1 = (2/L) * tx(1:floor(Nx/2)); % Premier tronçon croissant  
x2 = (-2/L) * tx(floor(Nx/2)+1:end) + 2; % Deuxième tronçon décroissant  
x = [x1, x2]; % Signal complet
```

```
figure;
plot(tx, x, 'b', 'LineWidth', 1.5);
title('Signal d'entrée triangulaire');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;
% Le signal triangulaire est généré et visualisé pour s'assurer qu'il est
conforme à la définition donnée.
```



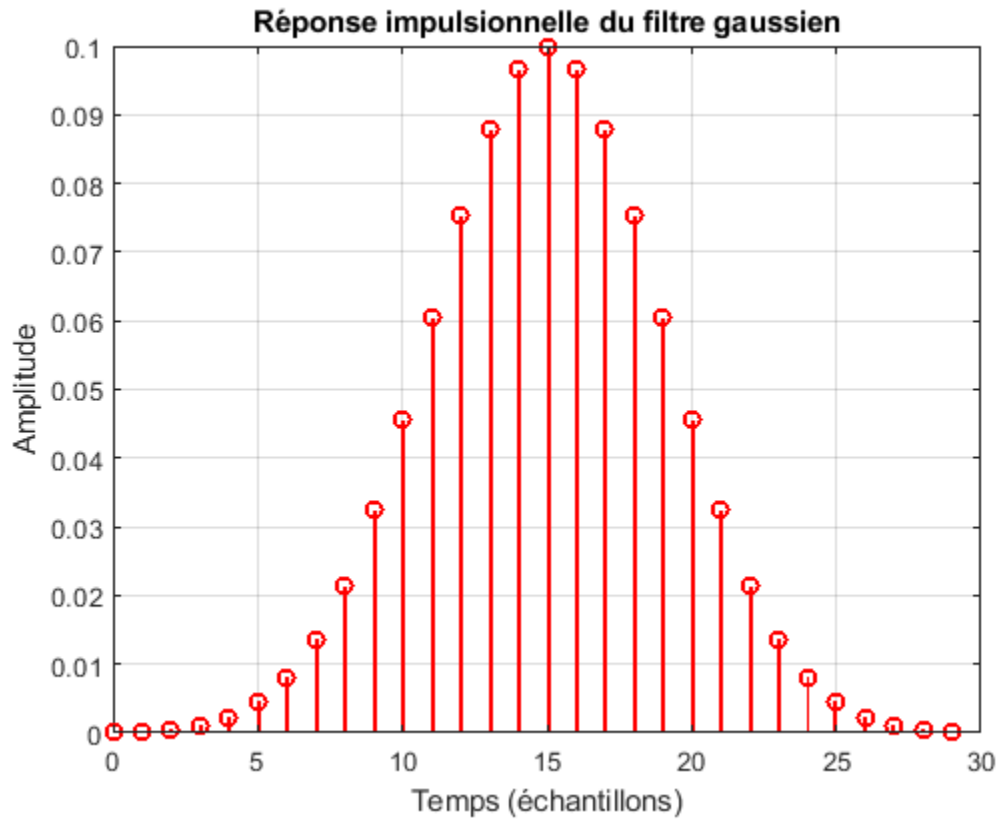
2. Simulation du filtre gaussien

```
m = 15; % Moyenne de la gaussienne
sigma = 4; % Écart type de la gaussienne
Nh = 30; % Horizon de la réponse impulsionnelle
th = 0:Nh-1; % Indices temporels du filtre

h = (1/(sigma*sqrt(2*pi))) * exp(-(th-m).^2 / (2*sigma^2)); % Réponse
impulsionnelle
grid on;

figure;
stem(th, h, 'r', 'LineWidth', 1.5);
title('Réponse impulsionnelle du filtre gaussien');
xlabel('Temps (échantillons)');
ylabel('Amplitude');
grid on;
```

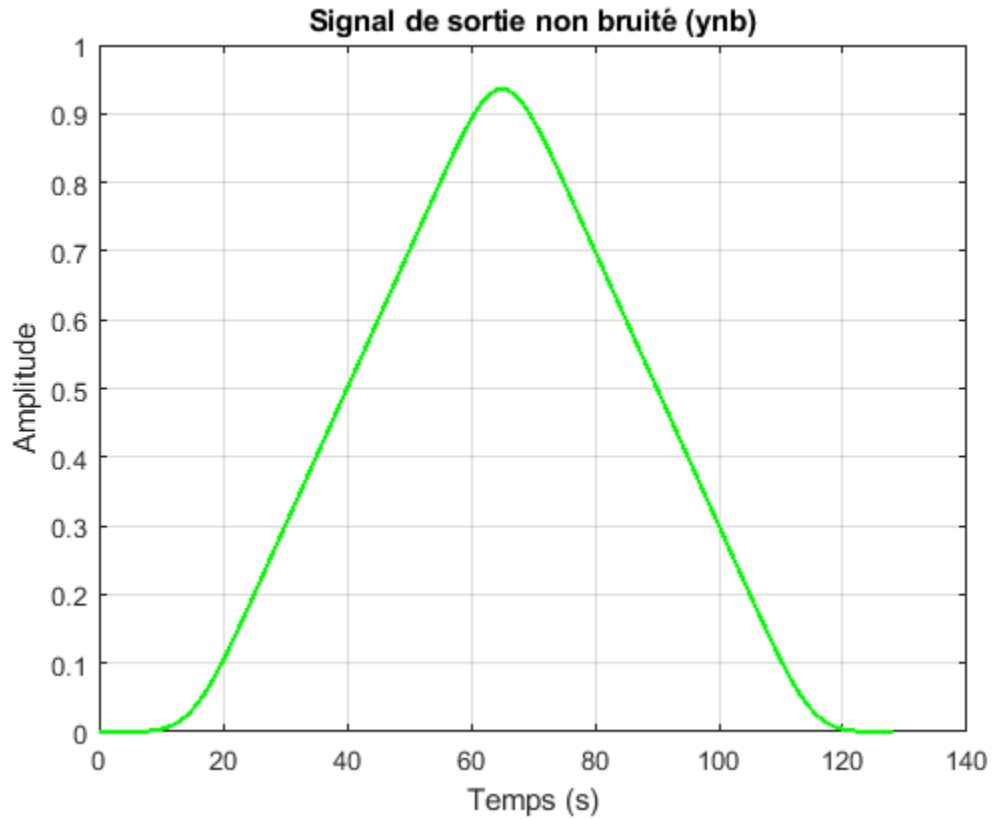
% La réponse impulsionnelle du filtre gaussien est correctement discrétisée et visualisée.



5. Simulation du problème direct

Convolution pour obtenir le signal de sortie non bruité

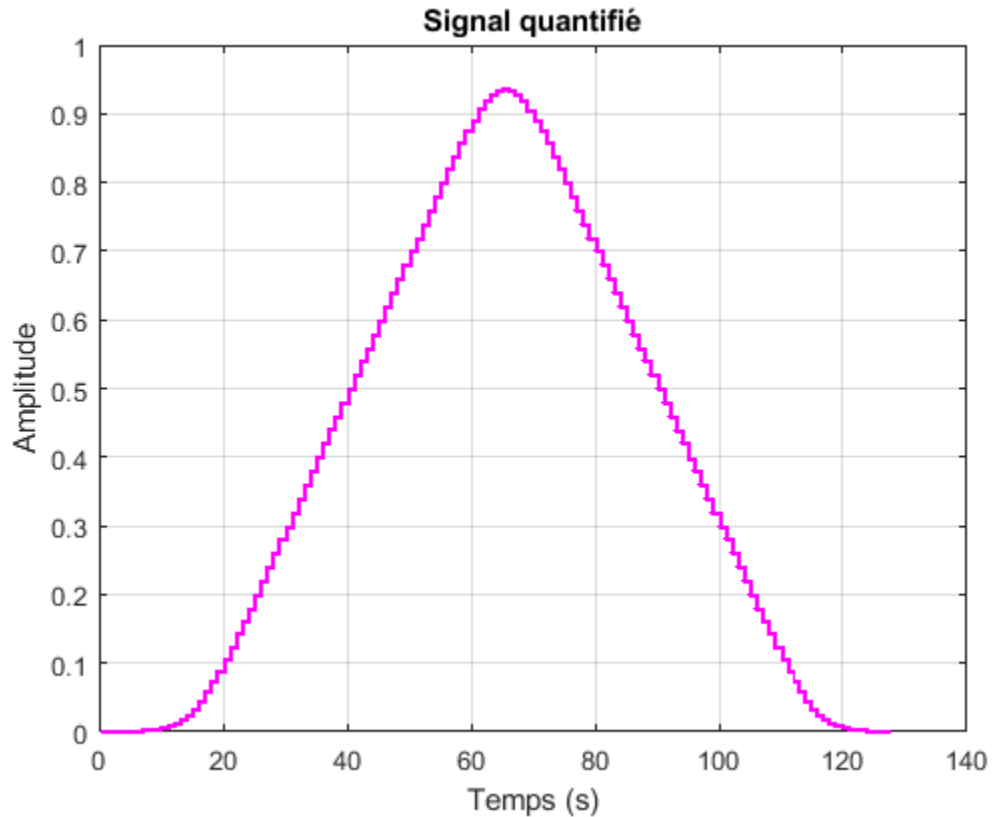
```
ynb = conv(x, h, 'full');  
ty = 0:length(ynb)-1; % Base de temps pour y  
  
figure;  
plot(ty, ynb, 'g', 'LineWidth', 1.5);  
title('Signal de sortie non bruité (ynb)');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;  
% La convolution est effectuée et le signal filtré est représenté. Il est  
lissé par l'effet du filtre gaussien.
```



6. Simulation du bruit de quantification

```
y = round(ynb * 2^10) / 2^10;
```

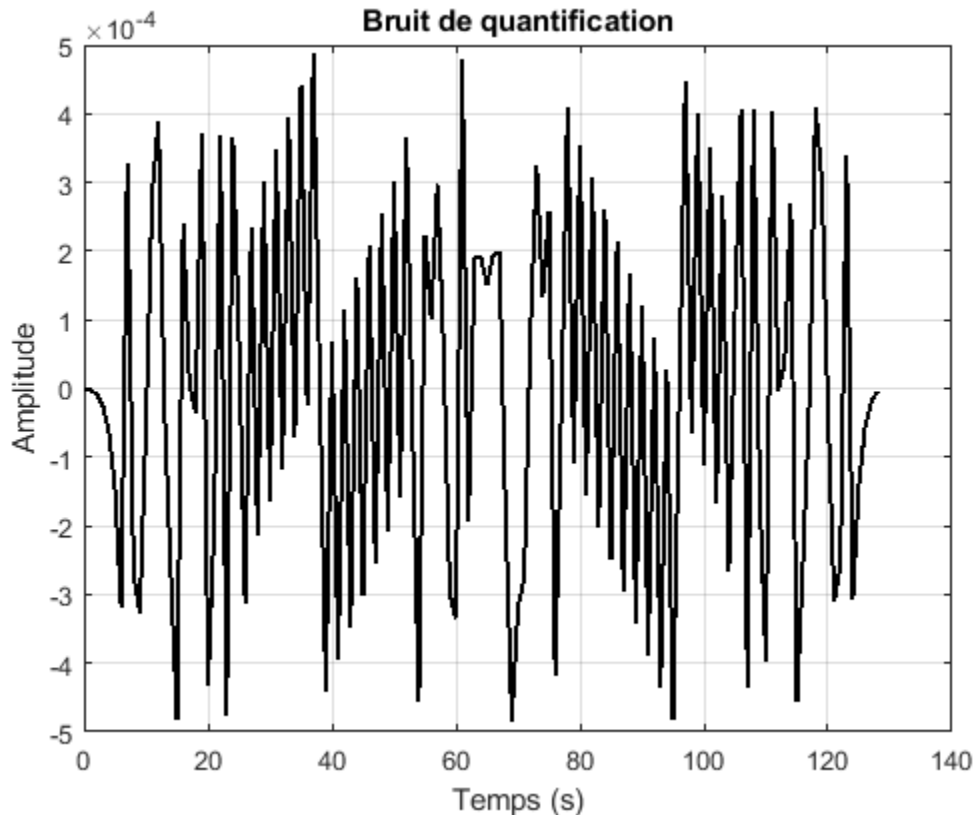
```
figure;  
stairs(ty, y, 'm', 'LineWidth', 1.5);  
title('Signal quantifié');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;
```



7. Calcul du bruit de quantification

```
b = y - ynb;
```

```
figure;  
plot(ty, b, 'k', 'LineWidth', 1.5);  
title('Bruit de quantification');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;  
% Le bruit de quantification est introduit pour simuler un convertisseur  
analogique-numérique.
```



8. Longueurs des signaux d'entrée et de la réponse impulsionnelle

```
Nx = length(x);
% Nombre d'échantillons du signal d'entrée x[n]
Nh = length(h);
% Nombre d'échantillons de la réponse impulsionnelle h[n]
% Calcul de la longueur du signal de sortie non bruité
Ny = Nx + Nh - 1;
% Affichage
fprintf('La dimension de y_{nb}[n] est : Ny= %d\n', Ny);
```

La dimension de $y_{nb}[n]$ est : $N_y = 129$

9. Dimensions

```
h = h(:);
Nx = length(x);
Nh = length(h);
% Nombre d'échantillons du signal d'entrée x[n]
% Nombre d'échantillons de la réponse impulsionnelle h[n]
Ny = Nx + Nh - 1; % Longueur du signal de sortie y[n]
% Construction de la première colonne de H
prem_col = [h; zeros(Ny - Nh, 1)]; % h suivi de zéros
```

```

% Construction de la première ligne de H
prem_lig = [h(1), zeros(1, Nx - 1)]; % h[0] suivi de zéros
% Construction de la matrice de convolution H
H = toeplitz(prem_col, prem_lig);
% Vérification des dimensions de H
fprintf('Dimensions de H : %d x %d\n', size(H, 1), size(H, 2));

```

Dimensions de H : 129 x 100

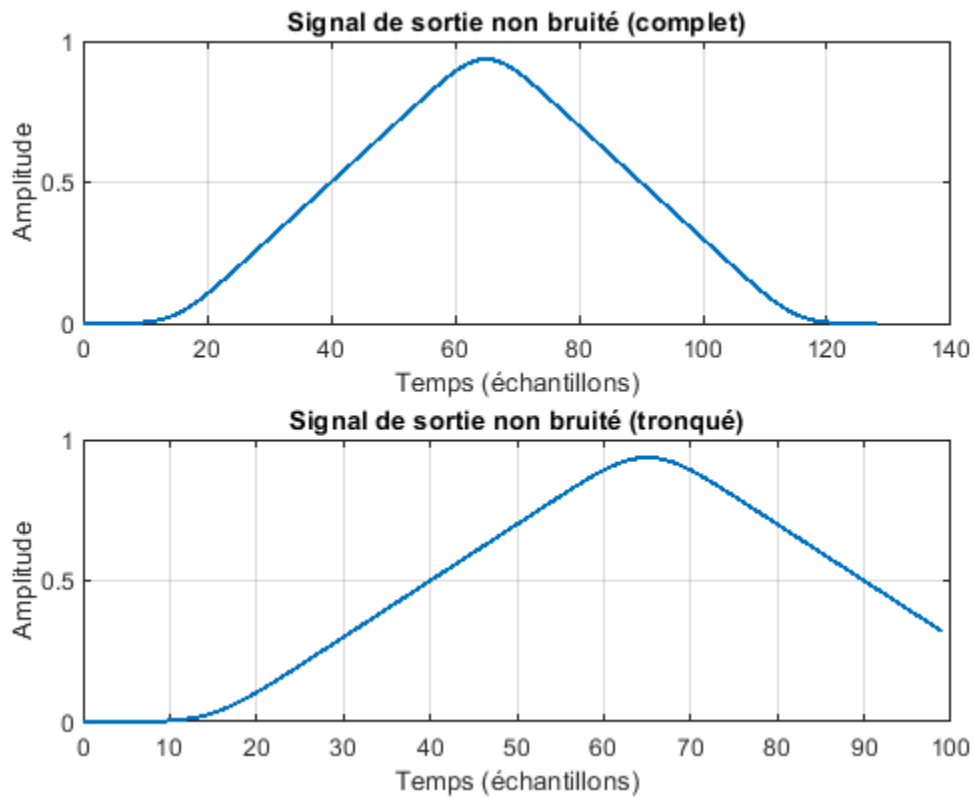
10. Calcul de la sortie non bruitée par opération matricielle

```

x=x(:);
% Calcul de la sortie non bruitée par opération matricielle
ynb = H * x;
% Application de l'erreur de troncature
ynb_truncated = ynb(1:Nx); % Tronquer pour correspondre à la longueur de
x[n]
% Affichage des résultats
figure;
% Signal non bruité (avant troncature)
subplot(2, 1, 1);
plot(0:length(ynb)-1, ynb, 'LineWidth', 1.5);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal de sortie non bruité (complet)');
grid on;
% Signal non bruité tronqué
subplot(2, 1, 2);
plot(0:Nx-1, ynb_truncated, 'LineWidth', 1.5);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal de sortie non bruité (tronqué)');
grid on;
% Vérification des dimensions
fprintf('Longueur de y_{nb}[n] (complet) : %d\n', length(ynb));
fprintf('Longueur de y_{nb}[n] (tronqué) : %d\n', length(ynb_truncated));

```

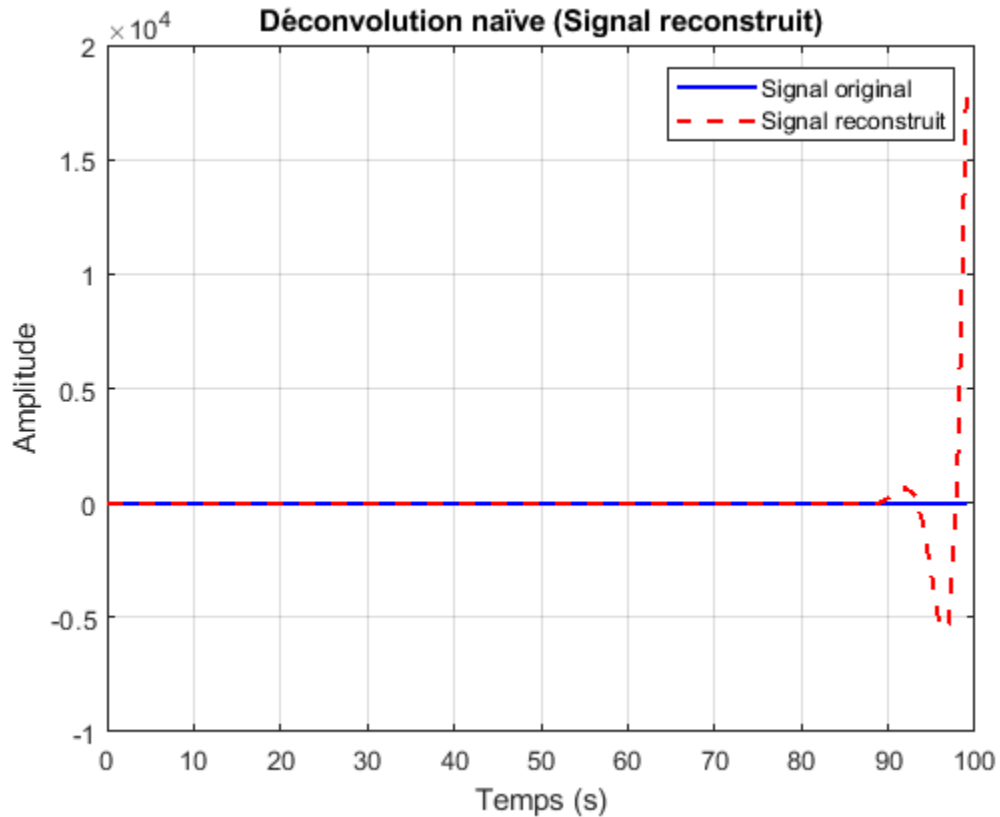
Longueur de $y_{nb}[n]$ (complet) : 129
Longueur de $y_{nb}[n]$ (tronqué) : 100



11. Déconvolution naïve et constat d'instabilité

Reconstruction du signal d'entrée à partir de la sortie non bruitée

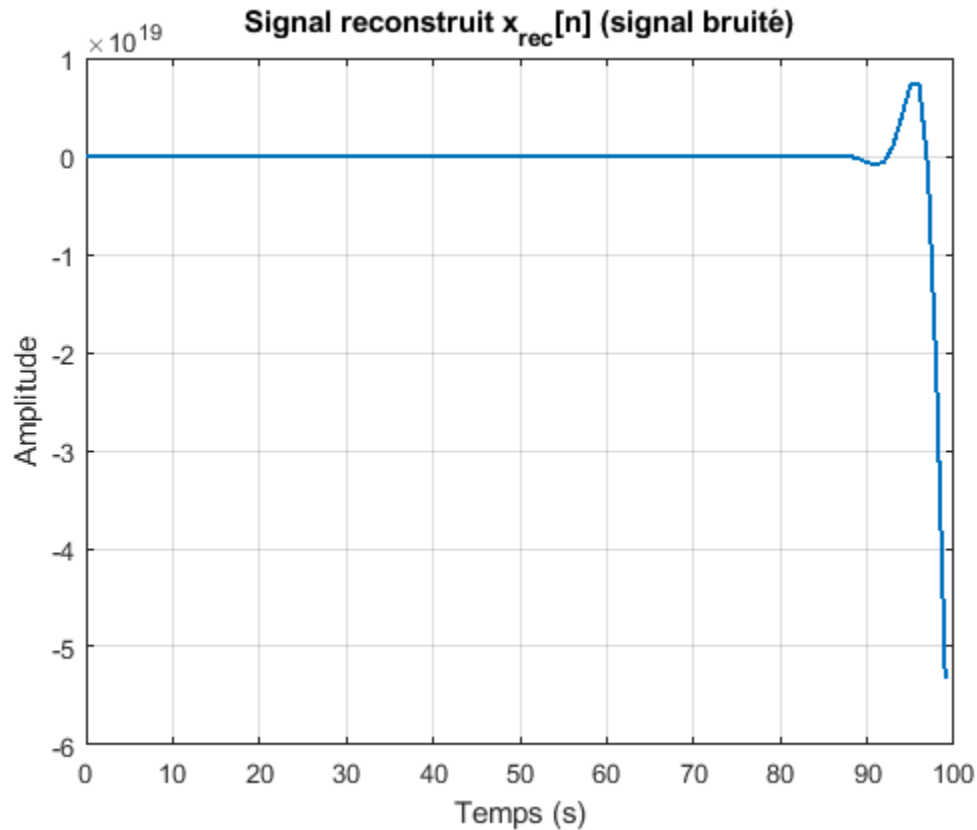
```
xrec_naive = deconv(y nb, h);  
  
figure;  
plot(tx, x, 'b', tx, xrec_naive(1:Nx), 'r--', 'LineWidth', 1.5);  
title('Déconvolution naïve (Signal reconstruit)');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
legend('Signal original', 'Signal reconstruit');  
grid on;  
% Le signal original est bien restauré lorsque les données ne sont pas  
bruitées.
```

2. Déconvolution

12. Base de temps pour le signal reconstruit

```
[x_rec_bruite, reste_bruite] = deconv(y, h);  
  
tx_rec_bruite = (0:length(x_rec_bruite)-1) * Te;  
% Représentation graphique du signal reconstruit  
figure;  
plot(tx_rec_bruite, x_rec_bruite, 'LineWidth', 1.5);  
xlabel('Temps (s)');  
ylabel('Amplitude');  
title('Signal reconstruit x_{rec}[n] (signal bruité)');  
grid on;
```



13.Initialisation des paramètres

```

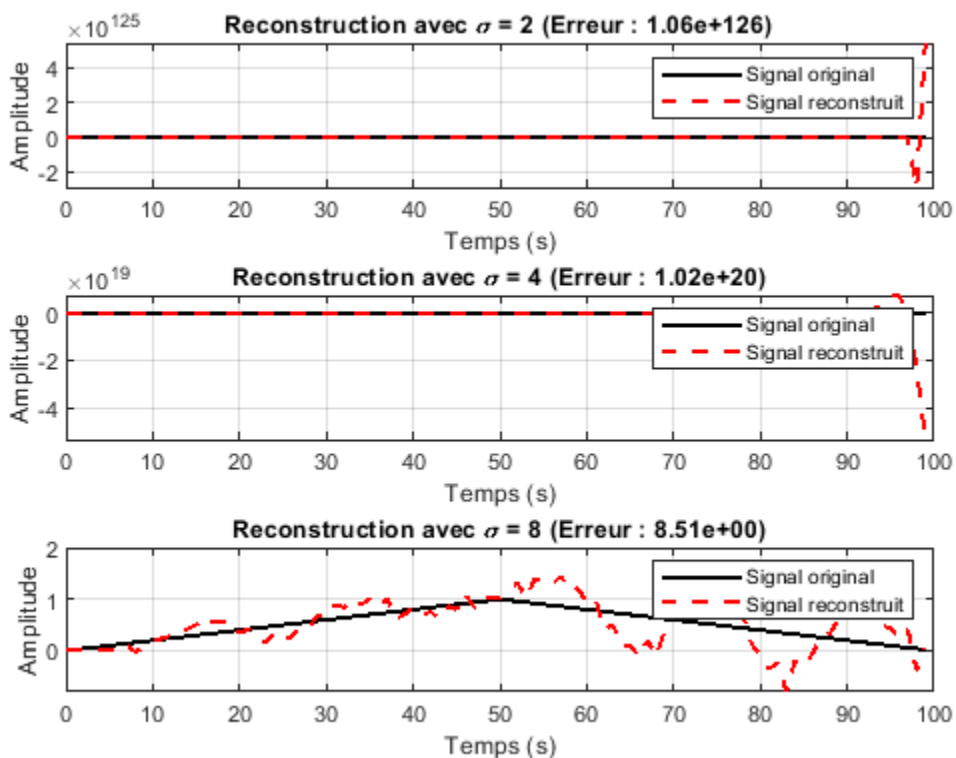
sigma_values = [2, 4, 8]; % Différentes valeurs de sigma
Nh = 30;
% Horizon temporel du filtre
reconstruction_errors = zeros(1, length(sigma_values)); % Erreurs de
reconstruction
figure;
for i = 1:length(sigma_values)
    sigma = sigma_values(i); % Changer la valeur de sigma
    % Recalcul de la réponse impulsionnelle h[n]
    n = 0:Nh-1;
% Indices temporels
    t = n * Te;
% Base de temps
    h = (1 / (sigma * sqrt(2 * pi))) * exp(-((t - m).^2) / (2 * sigma^2));
% Déconvolution "naïve" avec y bruité
    [x_rec, ~] = deconv(y, h);
% Base de temps pour x_rec
    tx_rec = (0:length(x_rec)-1) * Te;
% Calcul de l'erreur de reconstruction
    if length(x_rec) > length(x)
        x_rec = x_rec(1:length(x)); % Tronquer si nécessaire
    end
    reconstruction_error = norm(x - x_rec) / norm(x); % Erreur relative

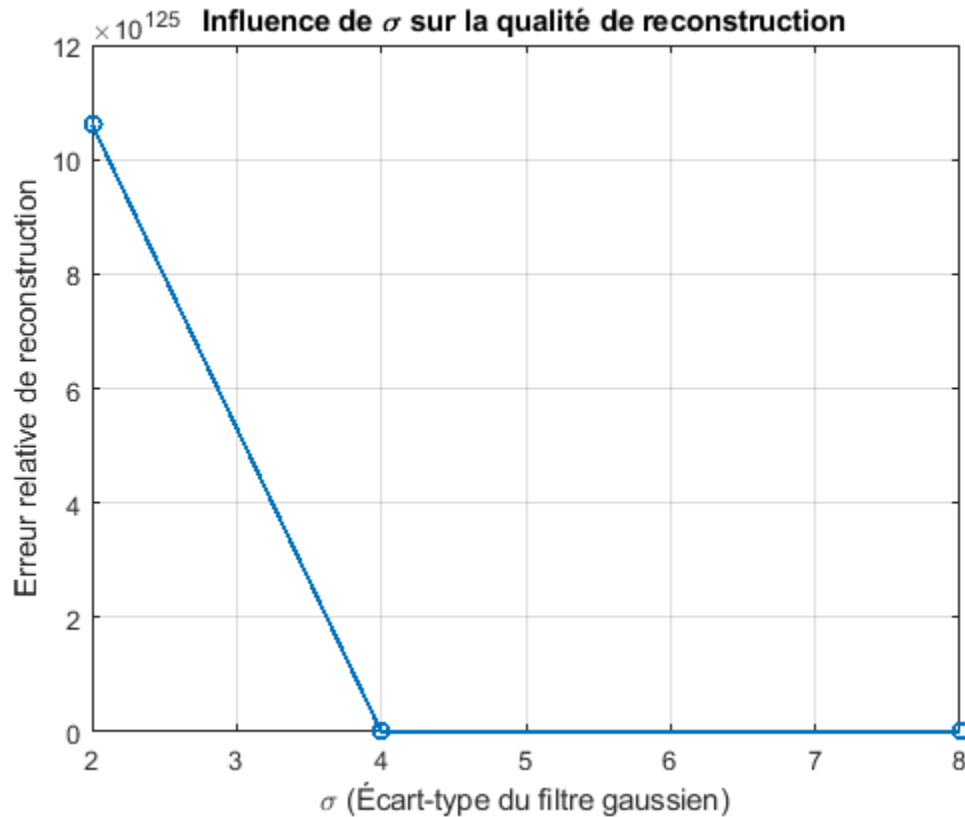
```

```

    reconstruction_errors(i) = reconstruction_error;
% Tracé du signal reconstruit
    subplot(length(sigma_values), 1, i);
    plot(tx, x, 'k', 'LineWidth', 1.5); hold on; % Signal original
    plot(tx_rec, x_rec, 'r--', 'LineWidth', 1.5); % Signal reconstruit
    xlabel('Temps (s)');
    ylabel('Amplitude');
    title(['Reconstruction avec \sigma = ', num2str(sigma), ' ...
' (Erreur : ', num2str(reconstruction_error, '%.2e'), ')']);
    legend('Signal original', 'Signal reconstruit');
    grid on;
end
% Affichage des erreurs de reconstruction
figure;
plot(sigma_values, reconstruction_errors, '-o', 'LineWidth', 1.5);
xlabel('\sigma (Écart-type du filtre gaussien)');
ylabel('Erreur relative de reconstruction');
title('Influence de \sigma sur la qualité de reconstruction');
grid on;

```





14. Construction de la matrice de convolution H

```
Ny = length(y nb); % Dimension du signal de sortie
col = zeros(Ny, 1);

col(1:length(h)) = h; % Insertion de h dans le premier vecteur colonne
lig=[h(1), zeros(1, Nx - 1)]; % la premier vecteur ligne
H=toeplitz(col,lig); % Matrice de convolution

% Reconstruction par moindres carrés
xrec_ls = (H' * H) \ (H' * y');

figure;
plot(tx, x, 'b', tx, xrec_ls, 'r--', 'LineWidth', 1.5);
title('Déconvolution par moindres carrés (Signal reconstruit)');
xlabel('Temps (s)');
ylabel('Amplitude');
legend('Signal original', 'Signal reconstruit');
grid on;

% Analyse du conditionnement de H
singular_values = svd(H);
condition_number = max(singular_values) /
min(singular_values(singular_values > 0));
```

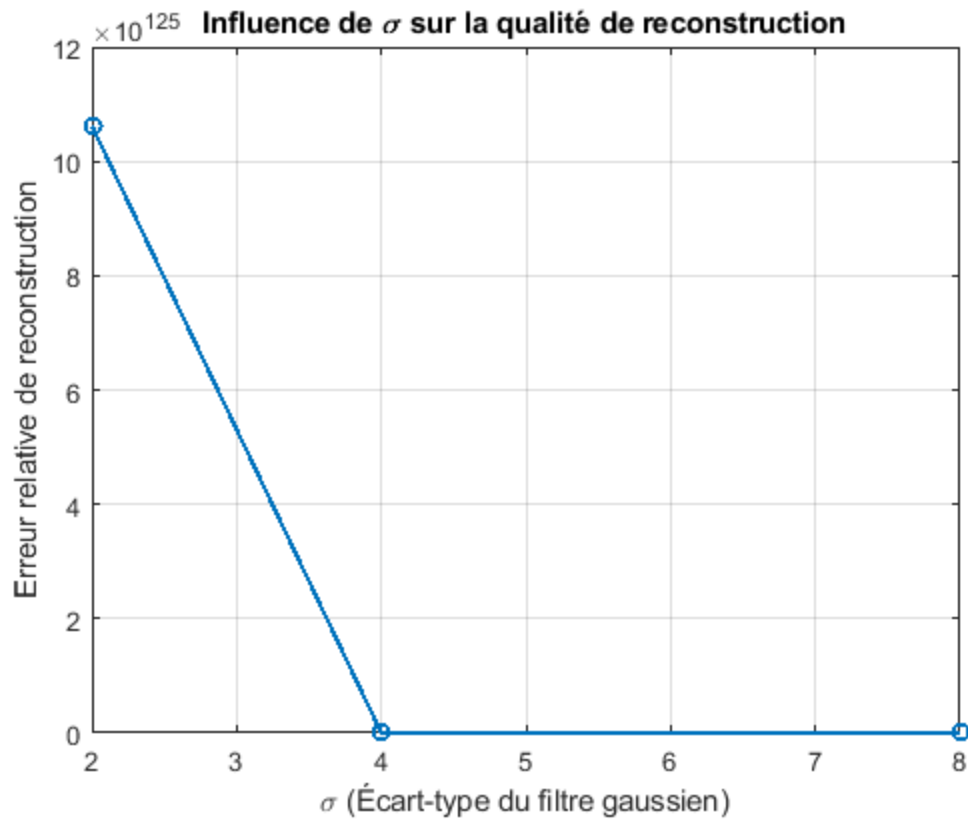
```

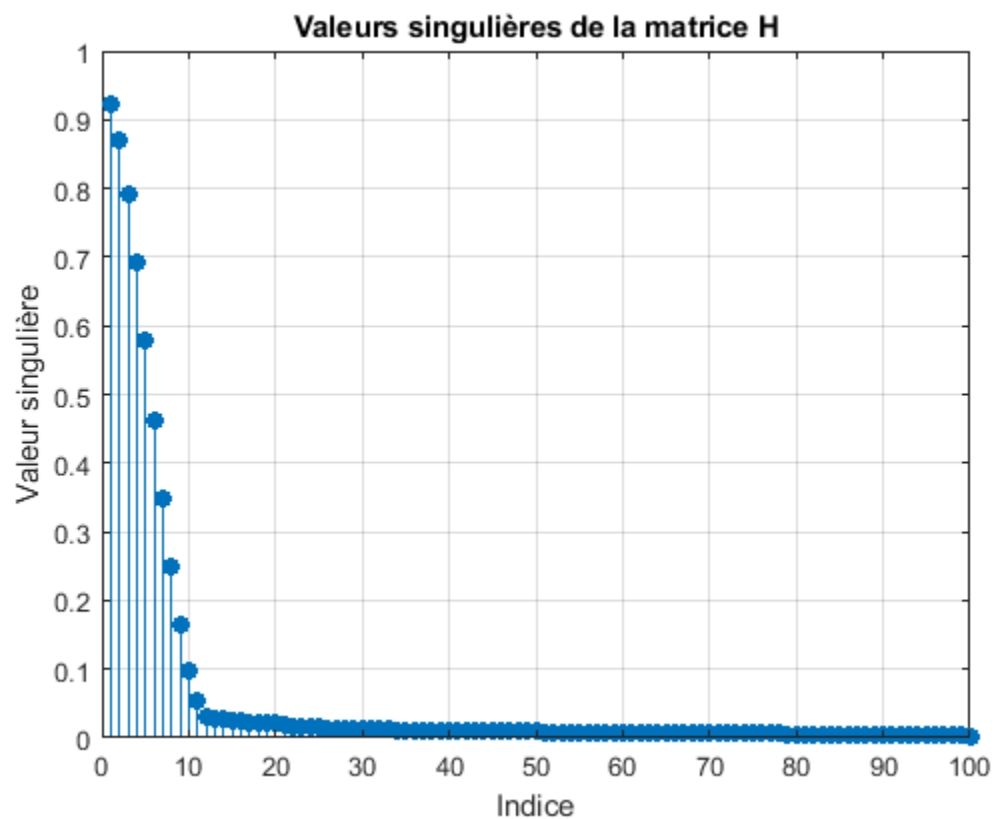
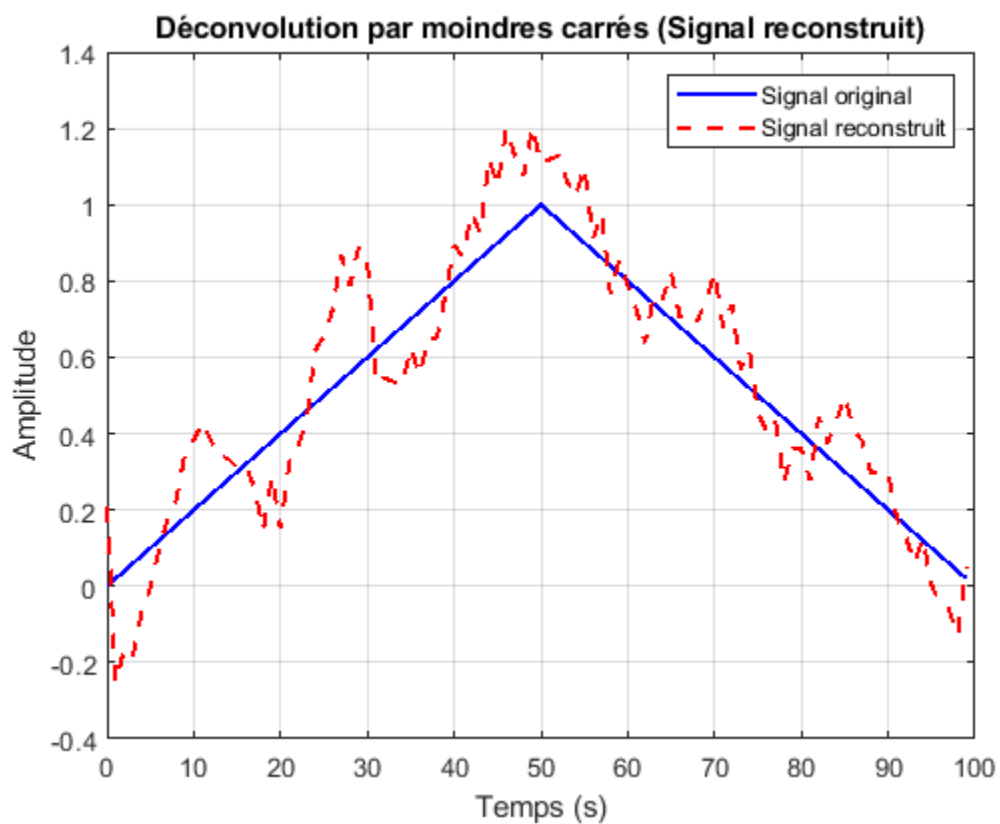
figure;
stem(singular_values, 'filled');
title('Valeurs singulières de la matrice H');
xlabel('Indice');
ylabel('Valeur singulière');
grid on;

fprintf('Le conditionnement de H est : %.2e\n', condition_number);

Le conditionnement de H est : 3.22e+02

```

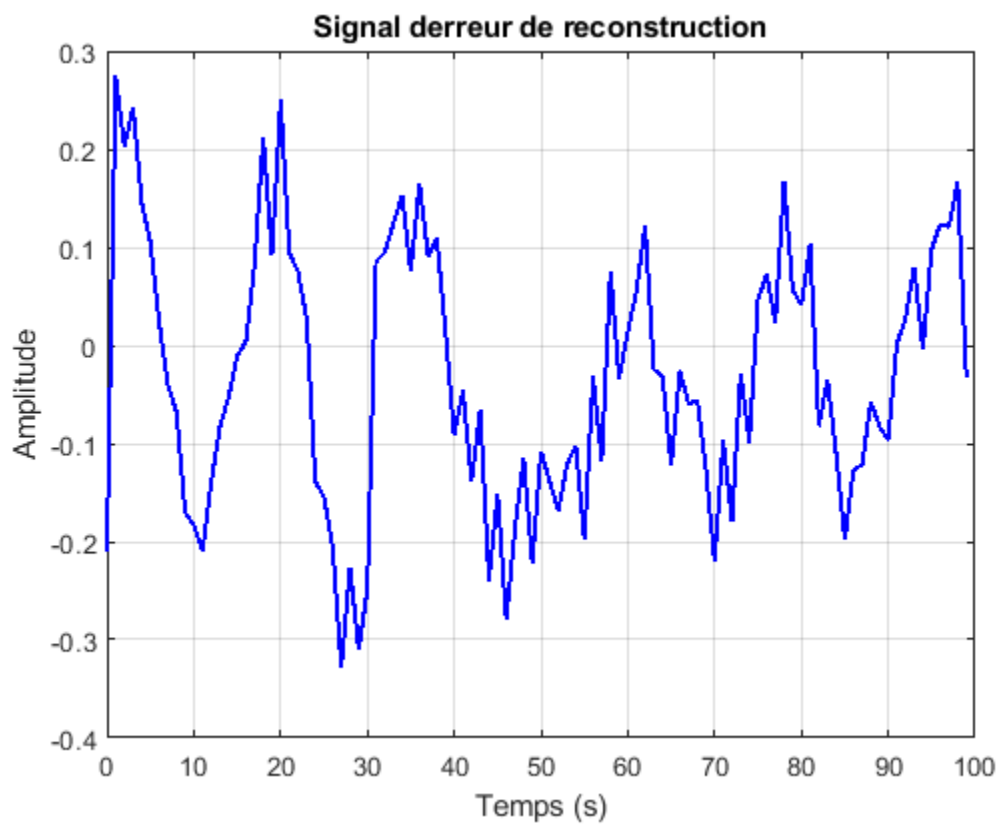
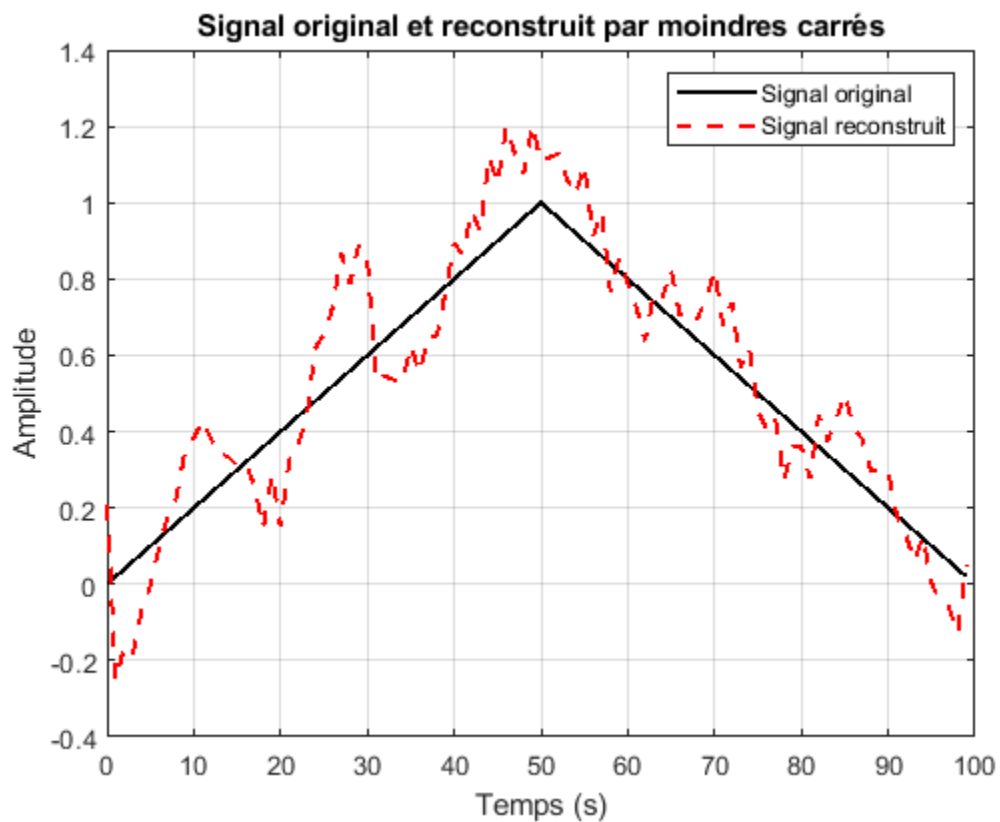




15. Tracé du signal original et reconstruit

```
figure;
plot(tx, x, 'k', 'LineWidth', 1.5); hold on; % Signal original
plot(tx, xrec_ls, 'r--', 'LineWidth', 1.5); % Signal reconstruit
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal original et reconstruit par moindres carrés');
legend('Signal original', 'Signal reconstruit');
grid on;
% Calcul de l'énergie de l'erreur de reconstruction
if length(xrec_ls) > length(x)
    xrec_ls = xrec_ls(1:length(x)); % Tronquer si nécessaire
end
error_signal = x - xrec_ls; % Signal d'erreur
error_energy = sum(error_signal.^2); % Énergie de l'erreur
% Affichage du résultat
fprintf('Énergie de l'erreur de reconstruction : %.5e\n', error_energy);
% Tracé du signal d'erreur
figure;
plot(tx, error_signal, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal d'erreur de reconstruction');
grid on;
```

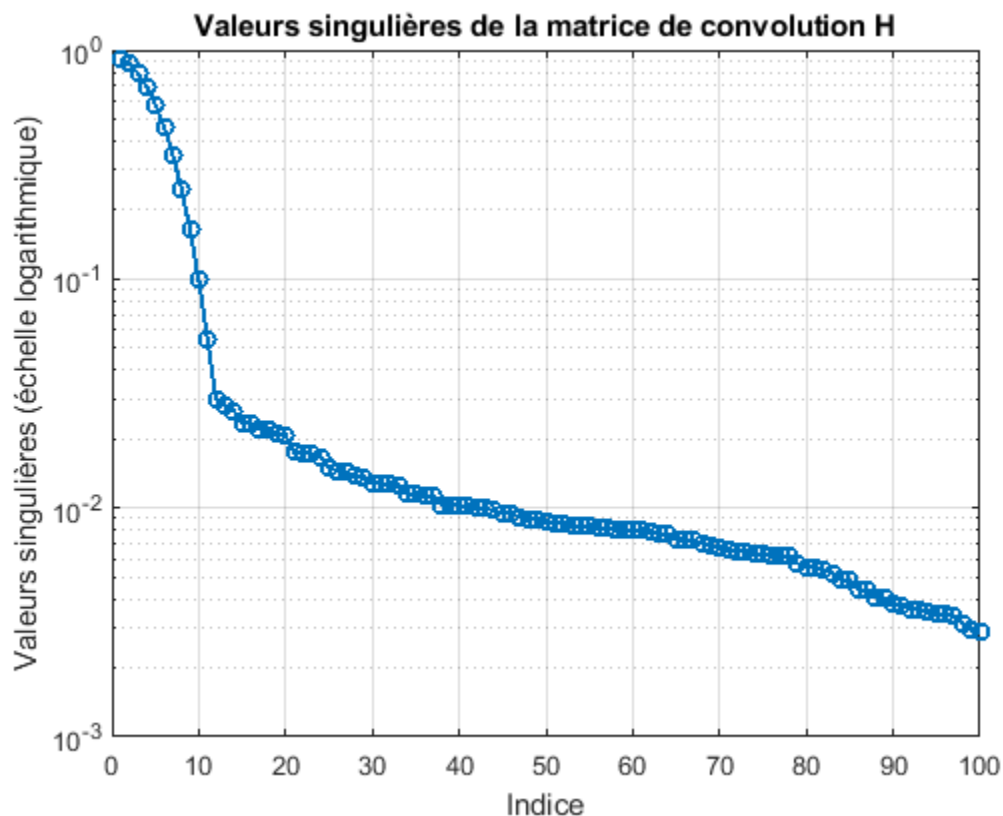
Énergie de l'erreur de reconstruction : 1.87596e+00



16. Calcul des valeurs singulières de la matrice de convolution H

```
singular_values = svd(H);  
% Calcul du conditionnement : ratio entre la plus grande et la plus petite  
valeur('singuliere non nulle')  
condition_number = max(singular_values)/min(singular_values(singular_values  
> eps));  
% Affichage du conditionnement  
fprintf('Conditionnement de H : %.2e\n', condition_number);  
% Tracé des valeurs singulières  
figure;  
semilogy(singular_values, 'o-', 'LineWidth', 1.5);  
xlabel('Indice');  
ylabel('Valeurs singulières (échelle logarithmique)');  
title('Valeurs singulières de la matrice de convolution H');  
grid on;
```

Conditionnement de H : 3.22e+02



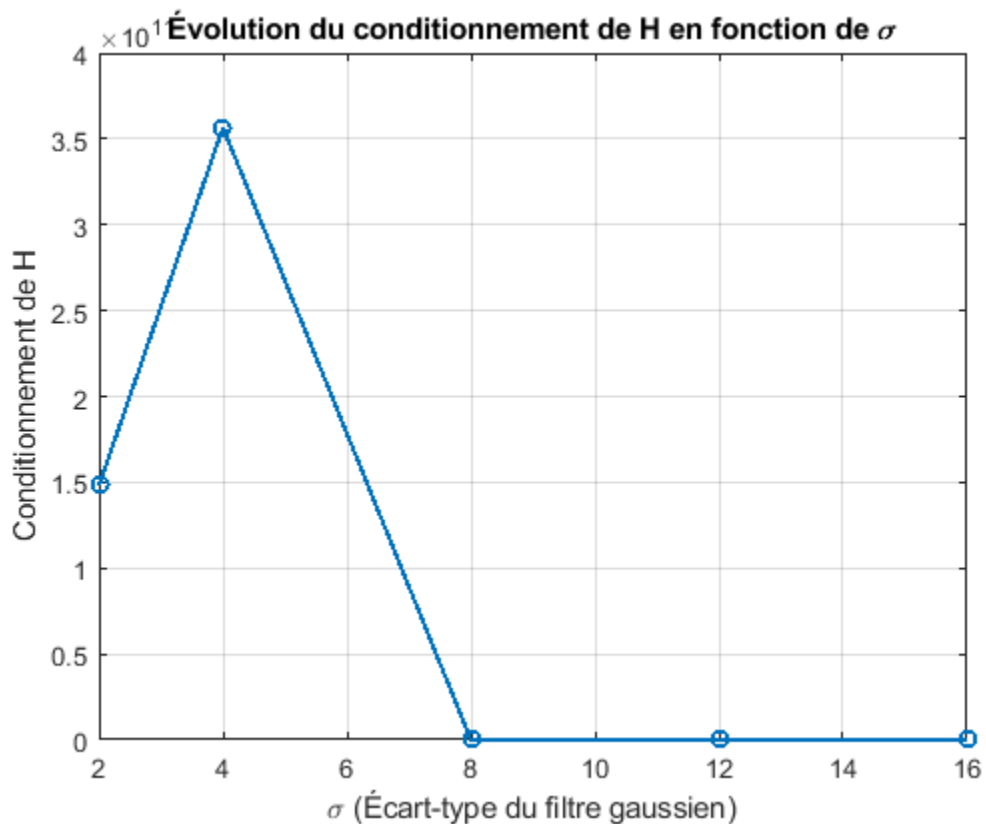
17. Initialisation des paramètres

```
sigma_values = [2, 4, 8, 12, 16]; % Différentes valeurs de sigma  
condition_numbers = zeros(1, length(sigma_values)); % Stocker les
```

```

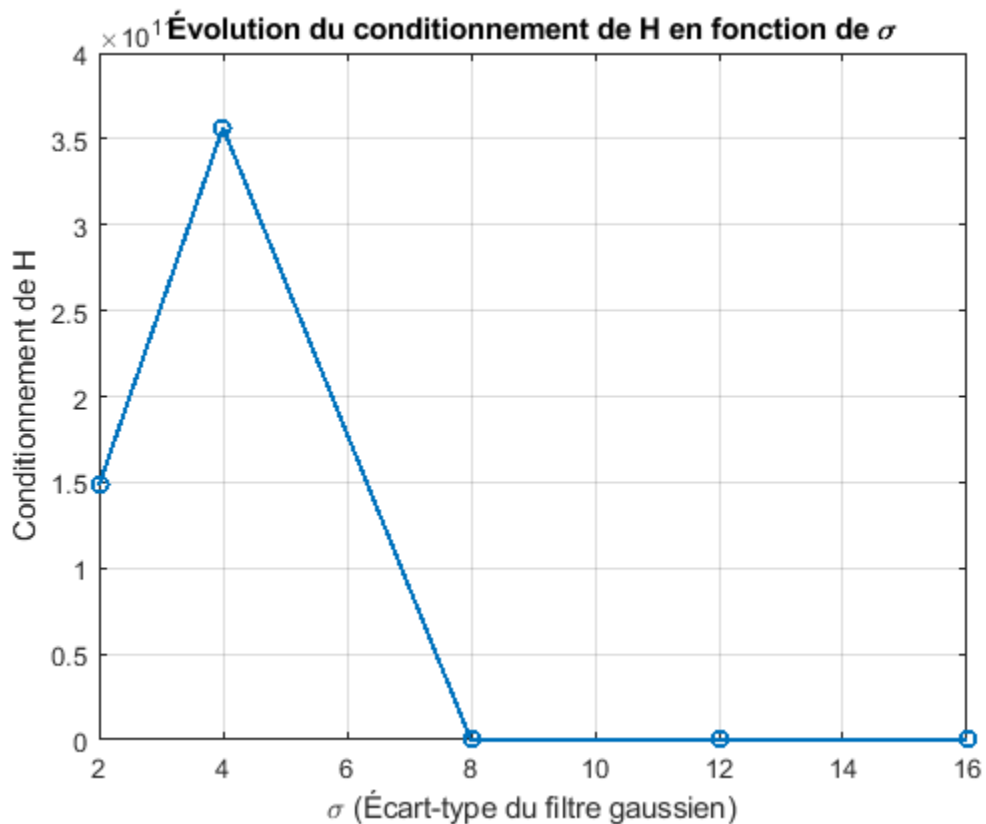
conditionnements
% Parcours des différentes valeurs de sigma
for i = 1:length(sigma_values)
    sigma = sigma_values(i);
% Calcul de la réponse impulsionnelle h[n] pour chaque sigma
    Nh = 30;
% Horizon temporel du filtre
    n = 0:Nh-1;
    t = n * Te;
    h = (1 / (sigma * sqrt(2 * pi))) * exp(-(t - m).^2) / (2 * sigma^2));
% Construction de la matrice de convolution H
    prem_col = [h(:); zeros(length(x) - length(h), 1)];
    prem_lig = [h(1); zeros(length(x) - 1, 1)];
    H = toeplitz(prem_col, prem_lig);
% Calcul des valeurs singulières
    singular_values = svd(H); % Conditionnement de H
    condition_numbers(i) = max(singular_values) /
min(singular_values(singular_values > eps));
end
% Affichage des conditionnements en fonction de sigma
figure;
plot(sigma_values, condition_numbers, '-o', 'LineWidth', 1.5);
xlabel('\sigma (Écart-type du filtre gaussien)');
ylabel('Conditionnement de H');
title('Évolution du conditionnement de H en fonction de \sigma');
grid on;

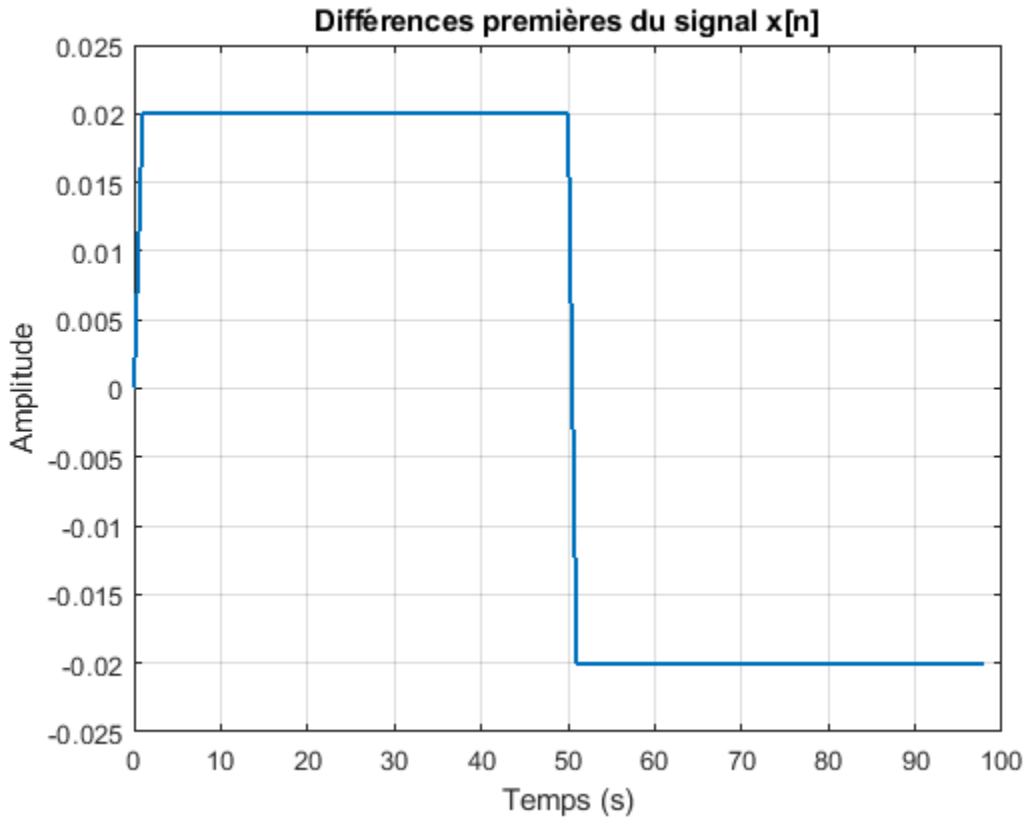
```



Vérification que x est un vecteur colonne

```
x = x(:); % Convertir en colonne si nécessaire
% Initialisation de la matrice D1
Nx = length(x); % Dimension de la matrice
dcol1 = zeros(1, Nx);
dcol1(1:2) = [1 -1]; % Valeurs pour les différences premières
dlig1 = zeros(1, Nx);
dlig1(1) = dcol1(1);
% Construction de la matrice D1 à l'aide de la fonction toeplitz
D1 = toeplitz(dcol1, dlig1);
% Calcul des différences premières
diff_premieres = D1 * x;
% Affichage du signal des différences premières
figure;
plot(tx(1:end-1), diff_premieres(1:end-1), 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Différences premières du signal x[n]');
grid on;
```





20.éconvolution par régularisation quadratique

Construction de la matrice D1 pour la régularisation

```
dcoll = zeros (1,Nx) ;
dcoll (1:2) = dcoll (1:2) + [1-1];
dlig1 = zeros (1,Nx);
dlig1 (1 ,1) = dcoll(1,1);
D1 = toeplitz (dcoll,dlig1);

% Paramètre de régularisation lambda
lambdas = logspace(-7, 2, 50); % Plage de valeurs pour lambda
errors = zeros(size(lambdas));
xrec_reg = zeros(Nx, length(lambdas));

for i = 1:length(lambdas)
    lambda = lambdas(i);
    xreg = ((H' * H) + lambda * (D1' * D1)) \ (H' * y');
    xrec_reg(:, i) = xreg;
    errors(i) = norm(x - xreg'); % Calcul de l'erreur de reconstruction
end
% Identifier la meilleure valeur de lambda
[~, idx] = min(errors);
best_lambda = lambdas(idx);

figure;
```

```

semilogx(lambdas, errors, 'b', 'LineWidth', 1.5);
title('Évolution de l erreur de reconstruction en fonction de \lambda');
xlabel('\lambda');
ylabel('Énergie de l erreur');
grid on;

% Reconstruction avec le meilleur lambda
xreg_best = xreg_reg(:, idx);

figure;
plot(tx, x, 'b', tx, xreg_best, 'r--', 'LineWidth', 1.5);
title('Déconvolution régularisée (Signal reconstruit)');
xlabel('Temps (s)');
ylabel('Amplitude');
legend('Signal original', 'Signal reconstruit');
grid on;

Error using *
Incorrect dimensions for matrix multiplication. Check that the number of
columns in the first matrix matches the number of rows in the second matrix.
To
operate on each element of the matrix individually, use TIMES (.* ) for
elementwise multiplication.

Error in untitled3 (line 330)
    xreg = ((H' * H) + lambda * (D1' * D1)) \ (H' * y');

```

Published with MATLAB® R2024a